

dataset

April 14, 2025

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
[2]: train = pd.read_csv('titanic_train.csv')
```

```
[3]: train.head()
```

```
[3]: PassengerId  Survived  Pclass  \
0             1         0         3
1             2         1         1
2             3         1         3
3             4         1         1
4             5         0         3
```

```
                                Name      Sex  Age  SibSp  \
0                Braund, Mr. Owen Harris   male  22.0     1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0     1
2                Heikkinen, Miss. Laina   female  26.0     0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)   female  35.0     1
4                Allen, Mr. William Henry   male  35.0     0
```

```
    Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN        S
1      0   PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0    113803  53.1000  C123        S
4      0    373450   8.0500   NaN        S
```

0.1 Missing Data

I have used seaborn to create a simple heatmap to see where we have missing data!

```
[4]: train.isnull()
```

```

[4]:
  PassengerId  Survived  Pclass   Name     Sex    Age  SibSp  Parch  Ticket \
0           False     False   False  False  False  False  False  False  False
1           False     False   False  False  False  False  False  False  False
2           False     False   False  False  False  False  False  False  False
3           False     False   False  False  False  False  False  False  False
4           False     False   False  False  False  False  False  False  False
5           False     False   False  False  False  True   False  False  False
6           False     False   False  False  False  False  False  False  False
7           False     False   False  False  False  False  False  False  False
8           False     False   False  False  False  False  False  False  False
9           False     False   False  False  False  False  False  False  False
10          False     False   False  False  False  False  False  False  False
11          False     False   False  False  False  False  False  False  False
12          False     False   False  False  False  False  False  False  False
13          False     False   False  False  False  False  False  False  False
14          False     False   False  False  False  False  False  False  False
15          False     False   False  False  False  False  False  False  False
16          False     False   False  False  False  False  False  False  False
17          False     False   False  False  False  True   False  False  False
18          False     False   False  False  False  False  False  False  False
19          False     False   False  False  False  True   False  False  False
20          False     False   False  False  False  False  False  False  False
21          False     False   False  False  False  False  False  False  False
22          False     False   False  False  False  False  False  False  False
23          False     False   False  False  False  False  False  False  False
24          False     False   False  False  False  False  False  False  False
25          False     False   False  False  False  False  False  False  False
26          False     False   False  False  False  True   False  False  False
27          False     False   False  False  False  False  False  False  False
28          False     False   False  False  False  True   False  False  False
29          False     False   False  False  False  True   False  False  False
..          ...      ...      ...      ...      ...      ...      ...      ...
861         False     False   False  False  False  False  False  False  False
862         False     False   False  False  False  False  False  False  False
863         False     False   False  False  False  True   False  False  False
864         False     False   False  False  False  False  False  False  False
865         False     False   False  False  False  False  False  False  False
866         False     False   False  False  False  False  False  False  False
867         False     False   False  False  False  False  False  False  False
868         False     False   False  False  False  True   False  False  False
869         False     False   False  False  False  False  False  False  False
870         False     False   False  False  False  False  False  False  False
871         False     False   False  False  False  False  False  False  False
872         False     False   False  False  False  False  False  False  False
873         False     False   False  False  False  False  False  False  False
874         False     False   False  False  False  False  False  False  False
875         False     False   False  False  False  False  False  False  False

```

876	False	False	False	False	False	False	False	False	False
877	False	False	False	False	False	False	False	False	False
878	False	False	False	False	False	True	False	False	False
879	False	False	False	False	False	False	False	False	False
880	False	False	False	False	False	False	False	False	False
881	False	False	False	False	False	False	False	False	False
882	False	False	False	False	False	False	False	False	False
883	False	False	False	False	False	False	False	False	False
884	False	False	False	False	False	False	False	False	False
885	False	False	False	False	False	False	False	False	False
886	False	False	False	False	False	False	False	False	False
887	False	False	False	False	False	False	False	False	False
888	False	False	False	False	False	True	False	False	False
889	False	False	False	False	False	False	False	False	False
890	False	False	False	False	False	False	False	False	False

	Fare	Cabin	Embarked
0	False	True	False
1	False	False	False
2	False	True	False
3	False	False	False
4	False	True	False
5	False	True	False
6	False	False	False
7	False	True	False
8	False	True	False
9	False	True	False
10	False	False	False
11	False	False	False
12	False	True	False
13	False	True	False
14	False	True	False
15	False	True	False
16	False	True	False
17	False	True	False
18	False	True	False
19	False	True	False
20	False	True	False
21	False	False	False
22	False	True	False
23	False	False	False
24	False	True	False
25	False	True	False
26	False	True	False
27	False	False	False
28	False	True	False
29	False	True	False

```

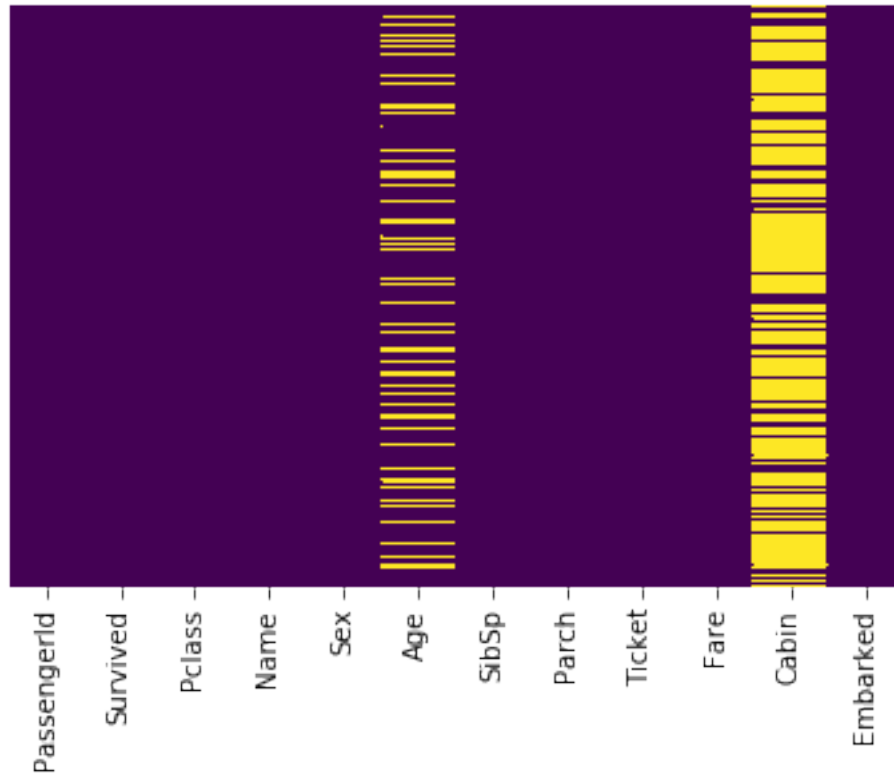
..      ...      ...      ...
861  False      True      False
862  False      False     False
863  False      True      False
864  False      True      False
865  False      True      False
866  False      True      False
867  False      False     False
868  False      True      False
869  False      True      False
870  False      True      False
871  False      False     False
872  False      False     False
873  False      True      False
874  False      True      False
875  False      True      False
876  False      True      False
877  False      True      False
878  False      True      False
879  False      False     False
880  False      True      False
881  False      True      False
882  False      True      False
883  False      True      False
884  False      True      False
885  False      True      False
886  False      True      False
887  False      False     False
888  False      True      False
889  False      False     False
890  False      True      False

```

```
[891 rows x 12 columns]
```

```
[10]: sns.heatmap(train.isnull(),yticklabels=False,cbar=False,cmap='viridis')
```

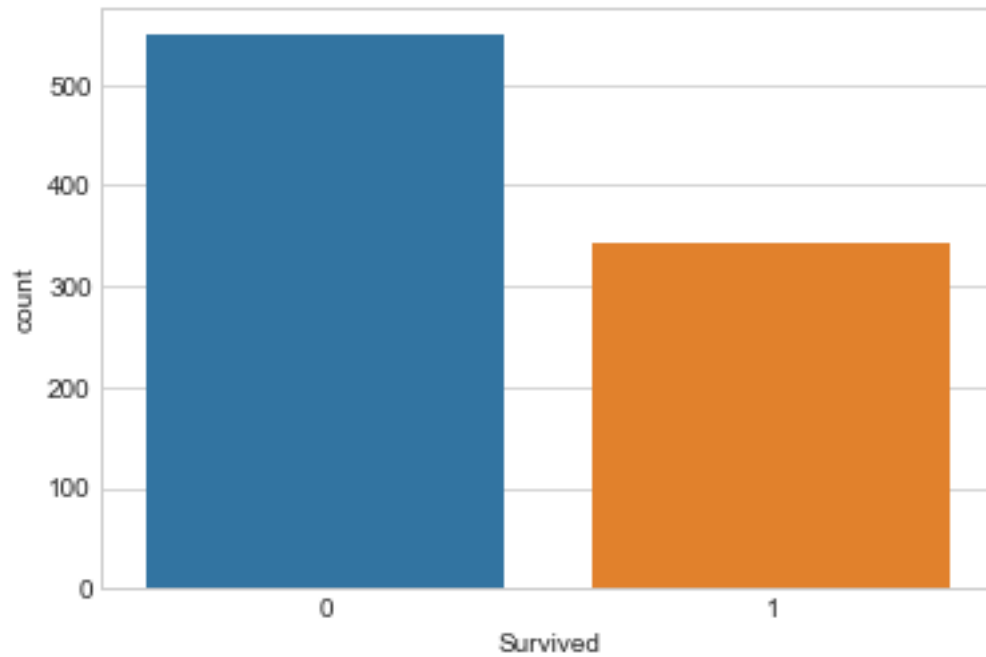
```
[10]: <matplotlib.axes._subplots.AxesSubplot at 0xc732278>
```



Roughly 20 percent of the Age data is missing. The proportion of Age missing is likely small enough for reasonable replacement with some form of imputation. Looking at the Cabin column, it looks like we are just missing too much of that data to do something useful with at a basic level.

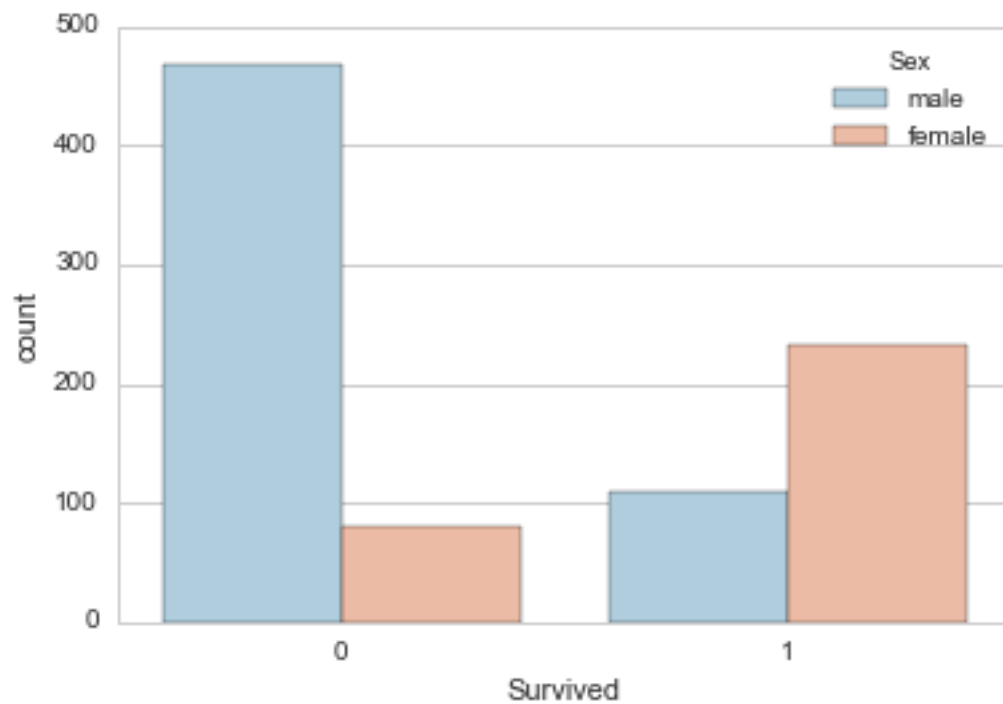
```
[11]: sns.set_style('whitegrid')
      sns.countplot(x='Survived',data=train)
```

```
[11]: <matplotlib.axes._subplots.AxesSubplot at 0xc6ebf98>
```



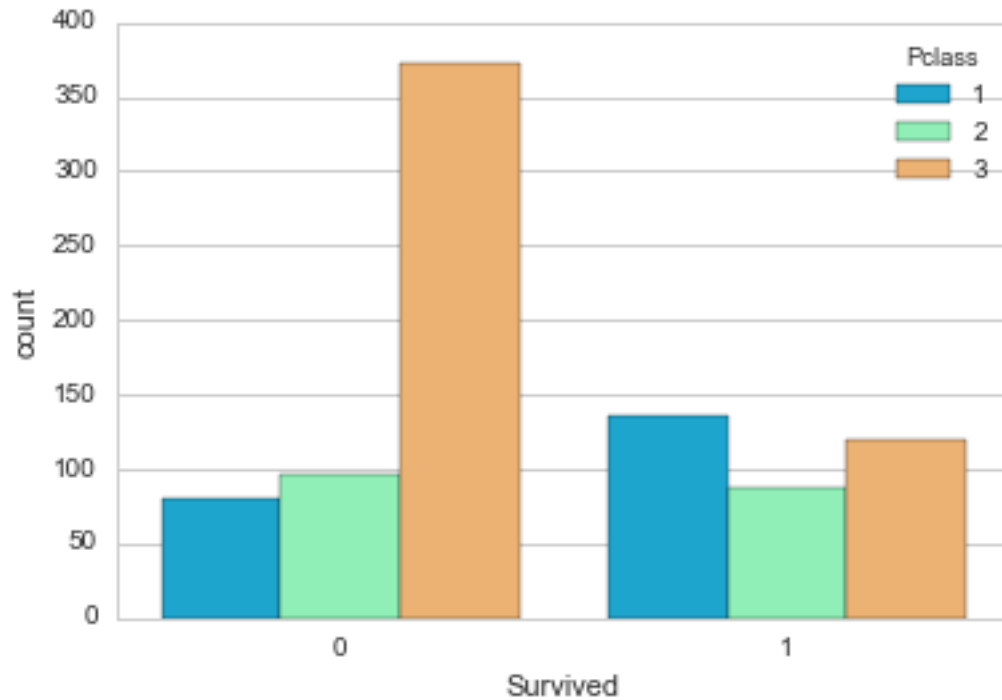
```
[78]: sns.set_style('whitegrid')
sns.countplot(x='Survived',hue='Sex',data=train,palette='RdBu_r')
```

```
[78]: <matplotlib.axes._subplots.AxesSubplot at 0x11b004a20>
```



```
[79]: sns.set_style('whitegrid')
sns.countplot(x='Survived',hue='Pclass',data=train,palette='rainbow')
```

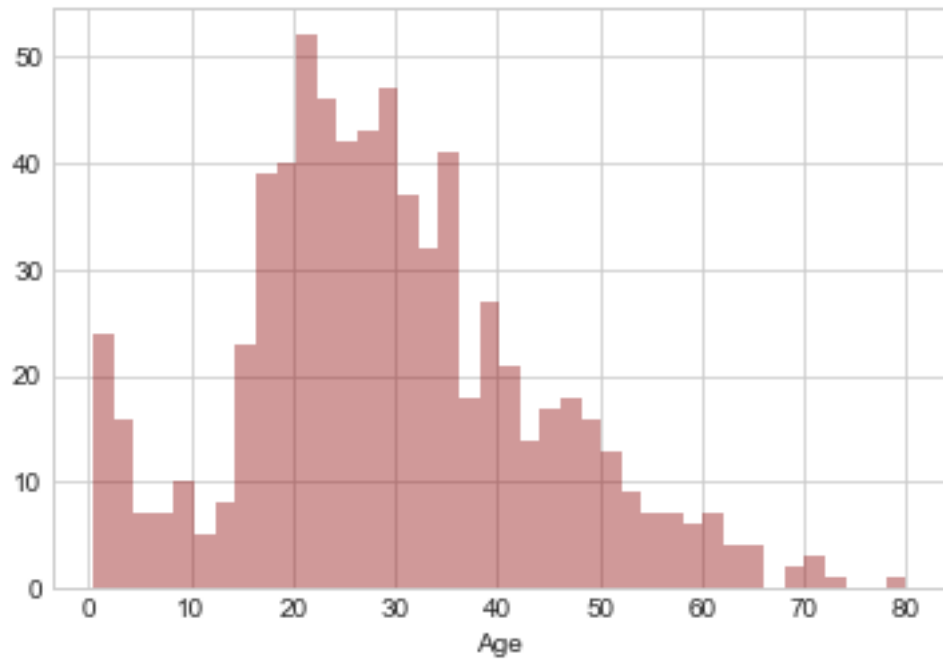
```
[79]: <matplotlib.axes._subplots.AxesSubplot at 0x11b130f28>
```



```
[22]: sns.distplot(train['Age'].dropna(),kde=False,color='darkred',bins=40)
```

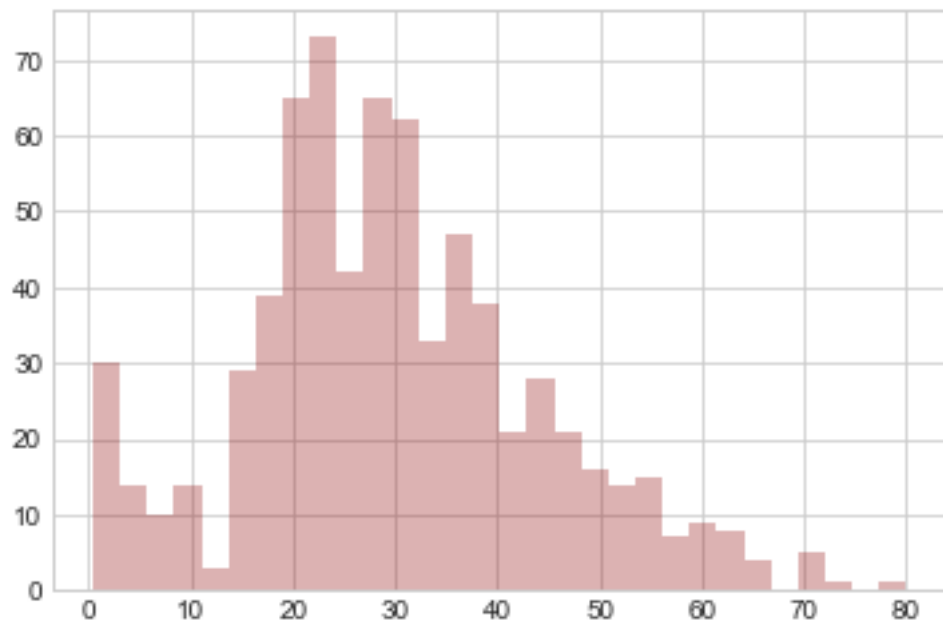
D:\anaconda\lib\site-packages\matplotlib\axes_axes.py:6462: UserWarning: The 'normed' kwarg is deprecated, and has been replaced by the 'density' kwarg.
warnings.warn("The 'normed' kwarg is deprecated, and has been "

```
[22]: <matplotlib.axes._subplots.AxesSubplot at 0xe0dea58>
```



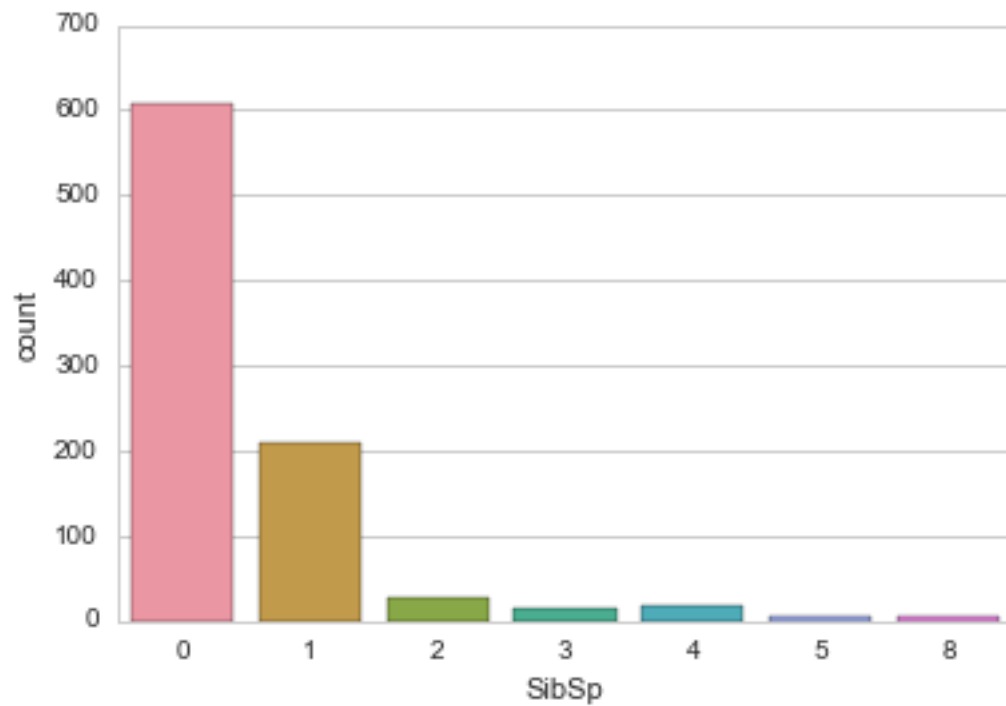
```
[25]: train['Age'].hist(bins=30,color='darkred',alpha=0.3)
```

```
[25]: <matplotlib.axes._subplots.AxesSubplot at 0xe2d8978>
```



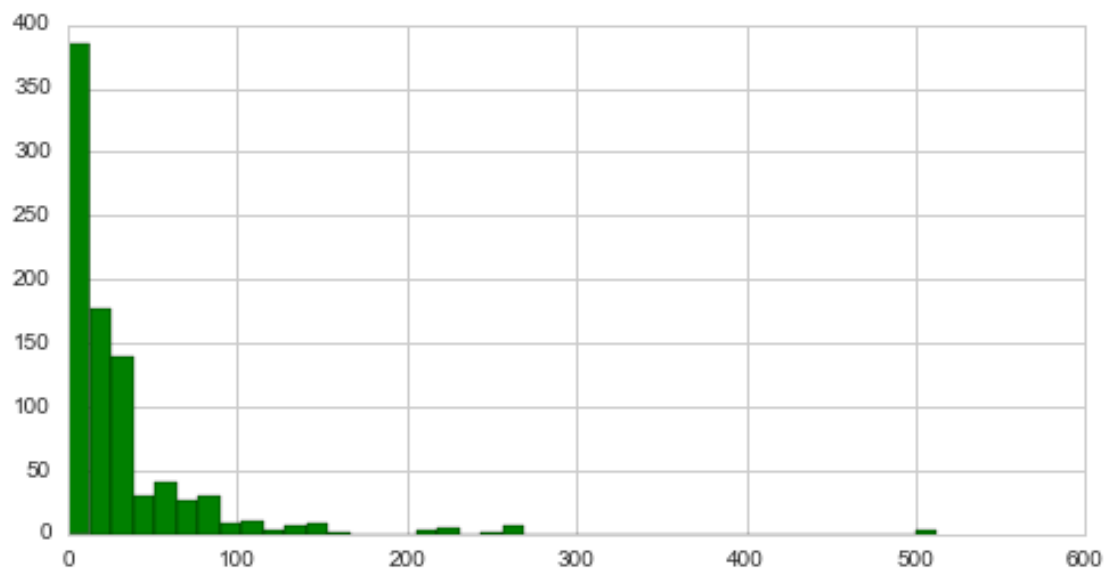

```
[82]: sns.countplot(x='SibSp',data=train)
```

```
[82]: <matplotlib.axes._subplots.AxesSubplot at 0x11c4139e8>
```



```
[83]: train['Fare'].hist(color='green',bins=40,figsize=(8,4))
```

```
[83]: <matplotlib.axes._subplots.AxesSubplot at 0x113893048>
```



0.1.1 Cufflinks for plots

Let's take a quick moment to show an example of cufflinks!

```
[84]: import cufflinks as cf
      cf.go_offline()
```

<IPython.core.display.HTML object>

```
[85]: train['Fare'].iplot(kind='hist',bins=30,color='green')
```

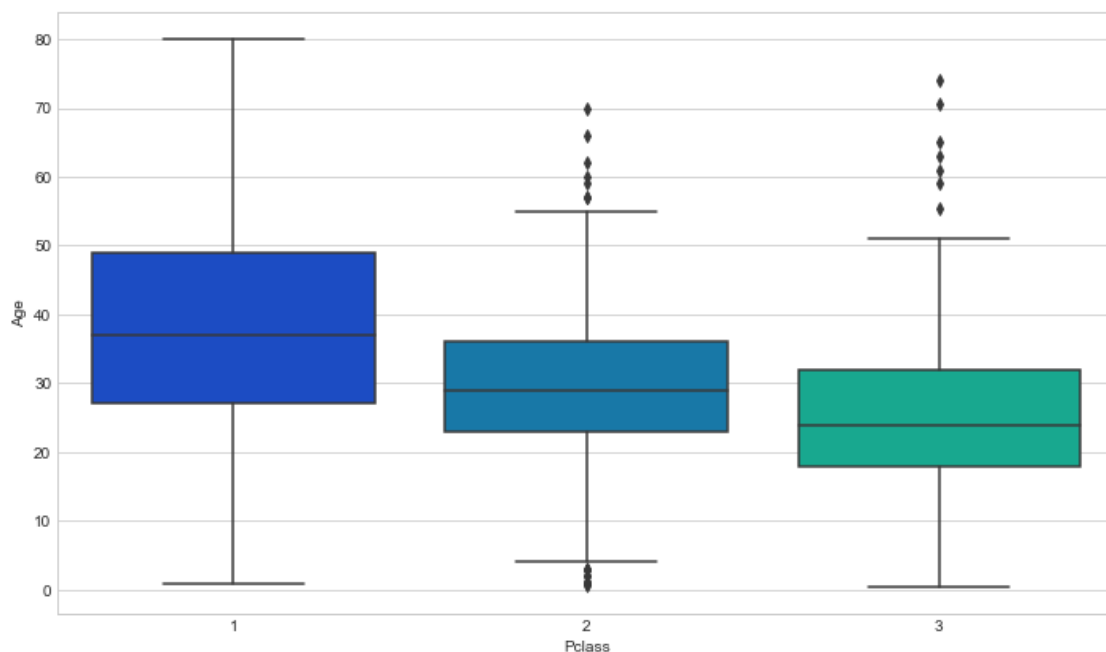
<IPython.core.display.HTML object>

0.2 Data Cleaning

I want to fill in missing age data instead of just dropping the missing age data rows.

```
[26]: plt.figure(figsize=(12, 7))
      sns.boxplot(x='Pclass',y='Age',data=train,palette='winter')
```

```
[26]: <matplotlib.axes._subplots.AxesSubplot at 0xe27f780>
```



We can see the wealthier passengers in the higher classes tend to be older, which makes sense. We'll use these average age values to impute based on Pclass for Age.

```
[27]: def impute_age(cols):  
    Age = cols[0]  
    Pclass = cols[1]  
  
    if pd.isnull(Age):  
  
        if Pclass == 1:  
            return 37  
  
        elif Pclass == 2:  
            return 29  
  
        else:  
            return 24  
  
    else:  
        return Age
```

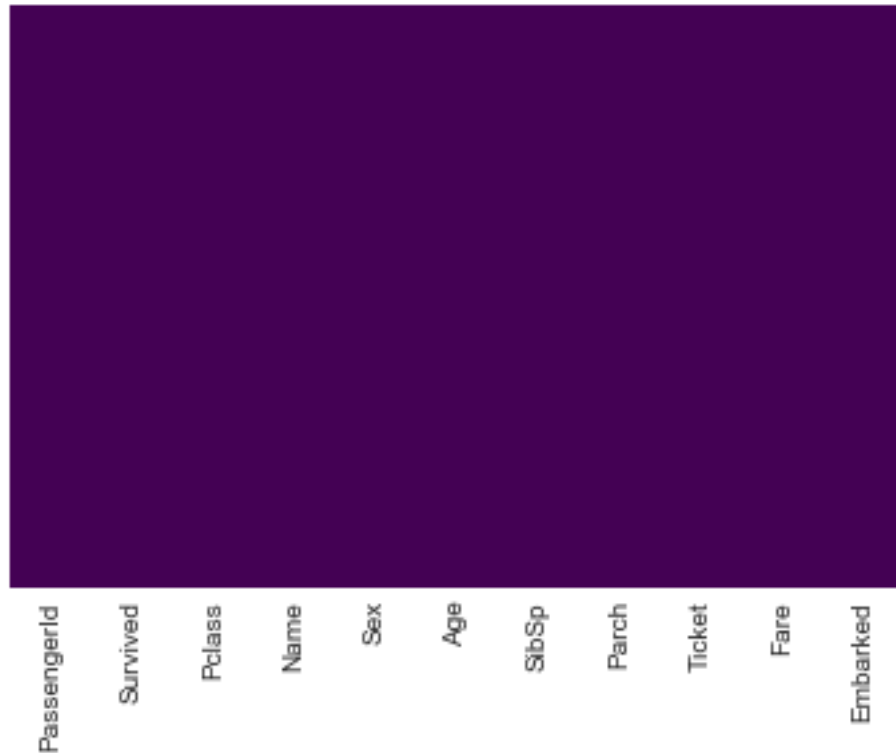
Now applying that function!

```
[28]: train['Age'] = train[['Age', 'Pclass']].apply(impute_age,axis=1)
```

Now let's check that heat map again!

```
[33]: sns.heatmap(train.isnull(),yticklabels=False,cbar=False,cmap='viridis')
```

```
[33]: <matplotlib.axes._subplots.AxesSubplot at 0xe4c27b8>
```



Great! Let's go ahead and drop the Cabin column and the row in Embarked that is NaN.

```
[30]: train.drop('Cabin',axis=1,inplace=True)
```

```
[91]: train.head()
```

```
[91]:
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Embarked
0	0	A/5 21171	7.2500	S
1	0	PC 17599	71.2833	C

2	0	STON/O2.	3101282	7.9250	S
3	0		113803	53.1000	S
4	0		373450	8.0500	S

```
[32]: train.dropna(inplace=True)
```

0.3 Converting Categorical Features

I'll need to convert categorical features to dummy variables using pandas! Otherwise it won't be able to directly take in those features as inputs.

```
[93]: train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 889 entries, 0 to 890
Data columns (total 11 columns):
PassengerId    889 non-null int64
Survived        889 non-null int64
Pclass         889 non-null int64
Name           889 non-null object
Sex            889 non-null object
Age           889 non-null float64
SibSp          889 non-null int64
Parch          889 non-null int64
Ticket         889 non-null object
Fare           889 non-null float64
Embarked       889 non-null object
dtypes: float64(2), int64(5), object(4)
memory usage: 83.3+ KB
```

```
[37]: pd.get_dummies(train['Embarked'],drop_first=True).head()
```

```
[37]:   Q  S
0  0  1
1  0  0
2  0  1
3  0  1
4  0  1
```

```
[38]: sex = pd.get_dummies(train['Sex'],drop_first=True)
embark = pd.get_dummies(train['Embarked'],drop_first=True)
```

```
[39]: train.drop(['Sex','Embarked','Name','Ticket'],axis=1,inplace=True)
```

```
[40]: train.head()
```

```
[40]:
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
0	1	0	3	22.0	1	0	7.2500
1	2	1	1	38.0	1	0	71.2833
2	3	1	3	26.0	0	0	7.9250
3	4	1	1	35.0	1	0	53.1000
4	5	0	3	35.0	0	0	8.0500

```
[96]: train = pd.concat([train,sex,embark],axis=1)
```

```
[97]: train.head()
```

```
[97]:
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare	male	Q	S
0	1	0	3	22.0	1	0	7.2500	1.0	0.0	1.0
1	2	1	1	38.0	1	0	71.2833	0.0	0.0	0.0
2	3	1	3	26.0	0	0	7.9250	0.0	0.0	1.0
3	4	1	1	35.0	1	0	53.1000	0.0	0.0	1.0
4	5	0	3	35.0	0	0	8.0500	1.0	0.0	1.0

```
[42]: train.drop('Survived',axis=1).head()
```

```
[42]:
```

	PassengerId	Pclass	Age	SibSp	Parch	Fare
0	1	3	22.0	1	0	7.2500
1	2	1	38.0	1	0	71.2833
2	3	3	26.0	0	0	7.9250
3	4	1	35.0	1	0	53.1000
4	5	3	35.0	0	0	8.0500

```
[43]: train['Survived'].head()
```

```
[43]:
```

0	0
1	1
2	1
3	1
4	0

Name: Survived, dtype: int64

```
[45]: from sklearn.model_selection import train_test_split
```

```
[46]: X_train, X_test, y_train, y_test = train_test_split(train.
↳drop('Survived',axis=1),
train['Survived'],
↳test_size=0.30,
random_state=101)
```

0.4 Training and Predicting

```
[47]: from sklearn.linear_model import LogisticRegression
```

```
[48]: logmodel = LogisticRegression()  
logmodel.fit(X_train,y_train)
```

```
[48]: LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,  
    intercept_scaling=1, max_iter=100, multi_class='ovr', n_jobs=1,  
    penalty='l2', random_state=None, solver='liblinear', tol=0.0001,  
    verbose=0, warm_start=False)
```

```
[49]: predictions = logmodel.predict(X_test)
```

```
[52]: from sklearn.metrics import confusion_matrix
```

```
[54]: accuracy=confusion_matrix(y_test,predictions)
```

```
[55]: accuracy
```

```
[55]: array([[144,  19],  
        [ 56,  48]], dtype=int64)
```

```
[57]: from sklearn.metrics import accuracy_score
```

```
[58]: accuracy=accuracy_score(y_test,predictions)  
accuracy
```

```
[58]: 0.7191011235955056
```

```
[50]: predictions
```

```
[50]: array([0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,  
        1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,  
        0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0,  
        0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1,  
        0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1,  
        0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1,  
        1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0,  
        0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0,  
        0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,  
        0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0,  
        1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,  
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,  
        0, 1, 0], dtype=int64)
```

0.5 Evaluation

We can check precision, recall, f1-score using classification report!

```
[104]: from sklearn.metrics import classification_report
```

```
[105]: print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
0	0.81	0.93	0.86	163
1	0.85	0.65	0.74	104
avg / total	0.82	0.82	0.81	267